Compiler Construction
Project Proposal



GeneScript : A Biology-Inspired Programming Language

Team Members:

Eman Aamir	23L-0612
Haleema Sadia	23L-0960
Ahsan Naveed	23L-0756

Instructor : Sir Faisal Aslam
Section: BCS-7G

Purpose

GeneScript is a beginner-friendly programming language designed for bioinformatics students. In most languages, DNA and RNA are ordinary text, so an invalid sequence like "ATXG", or an RNA sequence used where DNA belongs, runs without complaint. The mistake only shows up later as a wrong result, which is hard for a student to trace and teaches the wrong lesson.

GeneScript makes DNA and RNA built-in types with biology-based keywords such as `gene`, `express`, `report`, `stop` and `nucleus`. This helps students in three ways:

- **Errors are caught early.** The compiler rejects invalid sequences before the program runs, so students see the mistake and learn the biological rule behind it.
- **Code reads like the subject.** Reverse complement and transcription are built-in operations, so students spend their time on the algorithm and not on string handling.
- **Programs stay focused on biology.** Values and output are limited to sequences, so students practice working with biological data instead of general-purpose arithmetic.

Key Features

Novel syntax based on biology terms

GeneScript uses biology-inspired keywords and two distinctive expression forms: value-first declarations and assignments ("ATGC" -> dna g) and prefix concatenation (fuse a b). These forms give the language its own grammar and lexical rules.

Programming Concept	GeneScript Syntax
Function	`gene` (for example `gene merge(dna a, dna b)`)
Return	`express`
Variable	Value-first declaration, for example "ATGC" -> dna g or transcribe(g) -> rna r
Program entry point (main)	`gene nucleus()`, a gene with this reserved name and no parameters; execution starts here
Assignment operator	-> (value first, then destination; for example tail(g) -> g)
Comparison operators	= (equal), < (less than), > (greater than), <= (less than or equal), >= (greater than or equal). = compares two sequences of the

Programming Concept	GeneScript Syntax
	same type. The ordering operators compare single bases alphabetically
If / else	`when .. otherwise .. stop`
Else if	`otherwise when`
End of a block	`stop` (one per block; a whole `when .. otherwise .. stop` chain ends with a single `stop`)
Counted loop	`cycle i from a to b`
Loop over a sequence	`cycle i across s`
While	`grow until`
Break	`halt`
Continue	`skip`
Logical and	`both`
Logical or	`either`
Logical not	`lacks`
Boolean values (true / false)	`bound` / `unbound`
Sequence literal	`"ATGC"`, with `""` for the empty sequence
Base literal	`'G'`, a sequence of length one
Output (cout, printf)	`report`
Comment	`^^`

Core language

GeneScript includes what students need to write common algorithms:

- Functions, including recursion and mutual recursion (`gene`). A gene can call any gene in the file, whether it is defined before or after the call
- Conditionals (`when ... otherwise when ... otherwise ... stop`)
- Loops that walk through a sequence base by base (`cycle i across s`), counted loops (`cycle i from a to b`), and loops that repeat until a condition is met (`grow until`), with `halt` and `skip` for early exit and moving to the next base

- Variables that hold DNA or RNA, declared value-first with their type ("ATGC" -> dna g, transcribe(g) -> rna r)
- Conditions combined with `both`, `either` and `lacks`

The language is restricted to biological data: every value stored, passed to a gene, returned, or printed must be a DNA or RNA sequence (DNA: A, C, G, T; RNA: A, C, G, U). Integers are limited to loop bounds, indexes, sequence length (`#s`), and loop counters (`i`). In `cycle i from a to b`, bounds can only be integer literals, `#s`, or an enclosing loop counter. The loop runs from `a` up to (but excluding) `b`; `cycle i from 0 to #s` visits every base, while `cycle i across s` does the same for sequence `s`. Counters are read-only and exist only within their loops; integers cannot be declared, assigned, passed, or printed.

Built-in DNA and RNA types

DNA and RNA are separate built-in types, each with its own allowed letters:

- DNA may contain only A, C, G and T.
- RNA may contain only A, C, G and U.

Any other character, or a mix of the two alphabets, is a compile-time error. The compiler explains the error in biological terms, for example: "invalid base 'X' in DNA literal. DNA allows only A, C, G, T." Each message states the rule that was broken, which reinforces the biology.

How a literal gets its type. Sequences use double quotes (`"ATGC"`), while single bases use single quotes (`'G'`). A base is a sequence of length one, so both forms support the same operations. The empty sequence is `""`.

The compiler determines a literal's type from context, such as the destination type, parameter type, or another operand. Literals containing `T` are DNA, while those containing `U` are RNA. If the context is ambiguous, the default type is DNA. Empty sequences have no inherent type and rely on context. If a literal conflicts with its required type, such as `"AUG" -> dna x`, a compile-time error occurs.

Compiler Implementation

1. **Lexical Analysis (Flex):** Converts source code into tokens, including keywords, operators, sequence literals, identifiers, and comments.
2. **Parsing (Bison):** Checks syntax, including declarations, assignments, function calls, nested expressions, and block structure. It also validates loop-specific statements.
3. **Semantic Analysis:** Checks types, validates DNA/RNA sequences, resolves gene references, and ensures the program has exactly one valid entry point, `nucleus()`.

4. **Code Generation (LLVM):** Generates LLVM Intermediate Representation (IR), which is compiled ahead of time into a native executable that can be run to produce the program's output.

Biological Operations

Each operation below is built into the language, and the compiler checks that it is used on the right type. head, tail, s@i, # and fuse work on both DNA and RNA. ~ and transcribe work on DNA only.

Operation	Syntax	What it does
Head	`head(s)`	Returns the first base of a DNA or RNA sequence, as a sequence of length one. For s = "ATGC", `head(s)` returns "A". If s is empty, it returns "". This allows conditions such as `when head(a) = "A"`.
Tail	`tail(s)`	Returns the sequence without its first base. For s = "ATGC", `tail(s)` returns "TGC", and `tail("A")` returns "". If s is empty, it also returns "". This allows conditions such as `when tail(s) = ""`, which is true when s has at most one base, and recursion such as `merge(tail(a), b)`.
Indexing	`s@i`	Returns the base at position i, counting from 0, as a sequence of length one. For s = "ATGC": `s@0` is A, `s@1` is T, `s@2` is G, `s@3` is C. The position must be a `cycle` counter or an integer literal. Used to inspect a sequence one base at a time.
Length	`#s`	Returns the number of bases in s. `#"ATGC"` is 4. It can only be used as a loop bound (`cycle i from 0 to #s`); it cannot be stored, passed to a gene or reported. Used to control how far a loop runs.
Concatenation	fuse a b	Joins two sequences of the same type. A base is a sequence of length one, so a base can be joined to a sequence on either side. fuse "AT" "GC" gives "ATGC", and fuse head(s) tail(s) gives s. DNA with RNA is a compile-time error. Nested concatenations are

Operation	Syntax	What it does
		parenthesized, for example fuse (fuse a b) c. Used to build new sequences.
Reverse complement	`~d`	Swaps each base for its pair (A with T, C with G) and reverses the order. `~"ATGC"` gives "GCAT". This models the opposite strand of the DNA double helix.
Transcription	`transcribe(d)`	Replaces every T with U, converting DNA into its RNA equivalent. `transcribe("ATGC")` gives "AUGC".

Type Rules: The operations `s@i`, `head(s)`, and `tail(s)` preserve the sequence type. `fuse` joins two sequences of the same type; mixing DNA and RNA causes a compile-time error. The = operator compares sequences of the same type, while ordering operators (<, >, <=, >=) compare single bases alphabetically. The ~ and `transcribe` operations accept DNA only, as they represent DNA reverse complement and transcription. RNA reverse complement is outside the project's scope.

Sample Code 1: Filter G and C from a DNA sequence

sample.gene

```
gene gc_filter(dna seq)   ^^ keeps only the G and C bases of a DNA sequence
    "" -> dna result
    cycle i across seq
        when seq@i = 'G' either seq@i = 'C'
            fuse result seq@i -> result
        stop
    stop
    express result
stop

gene nucleus()
    "ATGCCATTGTA" -> dna g
    report gc_filter(g)
    report ~g

    transcribe(g) -> rna r
    report r
stop
```

Sample Output 1:

terminal - output

```
$ ./genec sample.gene && ./sample
GGCCG
TACAATGGCCAT
AUGGCCAUUGUA
$ _
```

Sample Code 2: Merge sort

merge_sort.gene

```
gene merge(dna a, dna b)   ^^ joins two sorted sequences into one
    when a = ""
        express b
    stop
    when b = ""
        express a
    stop
    when head(a) < head(b)
        express fuse head(a) merge(tail(a), b)
    otherwise
        express fuse head(b) merge(a, tail(b))
    stop
stop

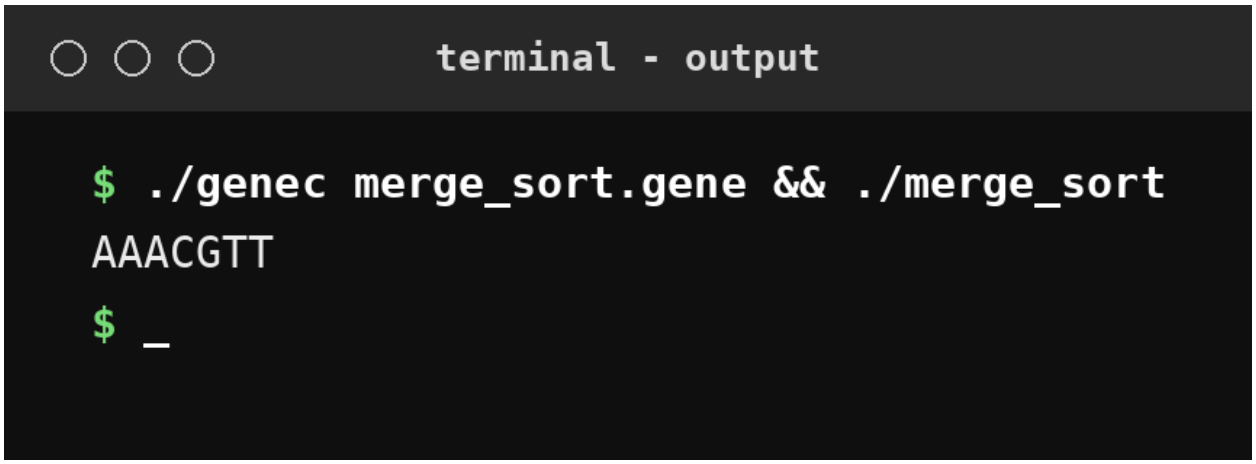
gene split_even(dna s)   ^^ bases at positions 0, 2, 4, ...
    when s = ""
        express s
    stop
    express fuse head(s) split_odd(tail(s))
stop

gene split_odd(dna s)   ^^ bases at positions 1, 3, 5, ...
    when s = ""
        express s
    stop
    express split_even(tail(s))
stop

gene merge_sort(dna s)   ^^ sorts the bases of a DNA sequence
    when tail(s) = ""
        express s
    stop
    express merge(merge_sort(split_even(s)), merge_sort(split_odd(s)))
stop

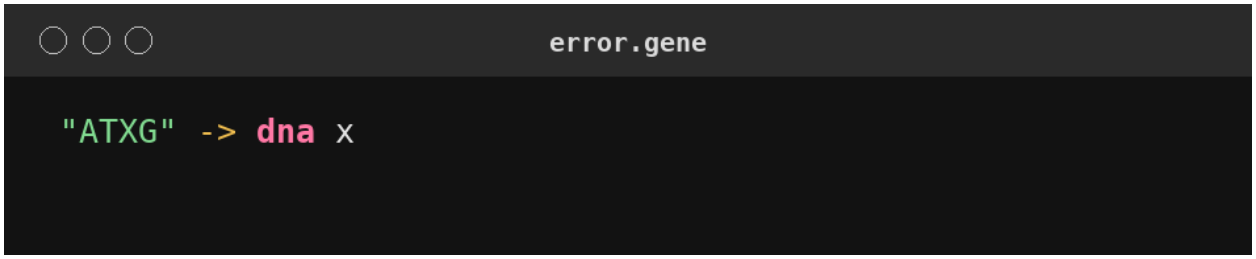
gene nucleus()
    "GATTACA" -> dna g
    report merge_sort(g)
stop
```

Sample Output 2:

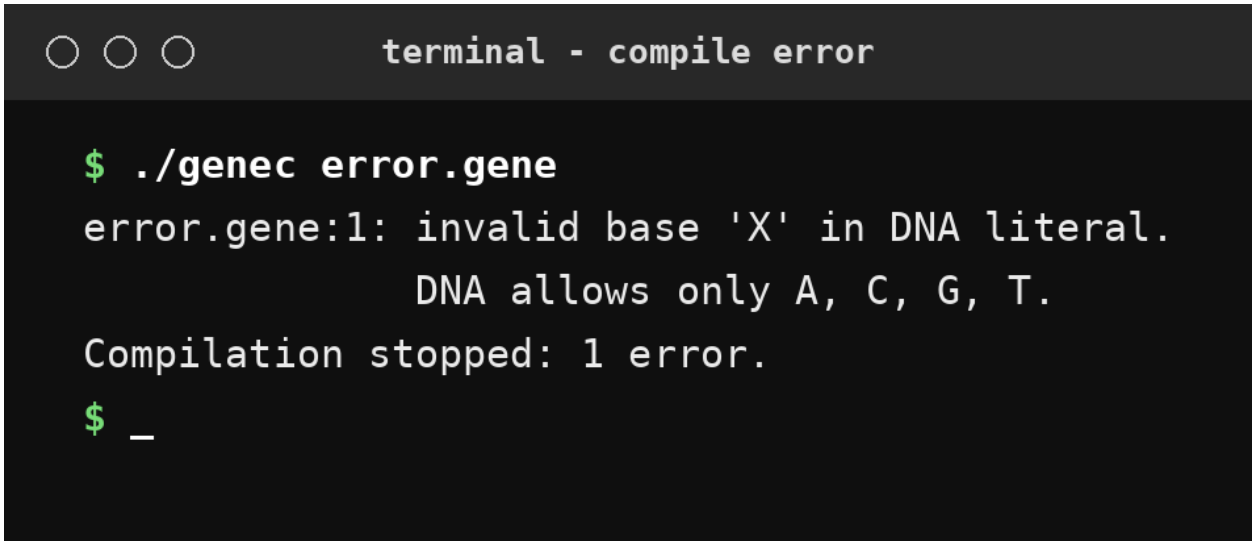


Sample Compile-Time Error

Source file (error.gene):



Compiler output:



Future Work

Translating RNA into proteins by codon is a natural next step. It is outside the current A/C/G/T/U restriction, so it is listed as a possible extension.